COLLEGE PROJECT DOCUMENTATION

LEGALEASE

AI-Powered Legal Document Generator

Project Title	LegalEase – AI-Powered Legal Document Generator
Technology	Python, Streamlit, FastAPI, Google Gemini
Application Type	Web-based AI-assisted legal document drafting application
Project Version	1.0.0

Student Name: _____

Register Number: _____

Department: _____

College: _____

Academic Year: _____

Prepared for academic project submission

CERTIFICATE

This is to certify that the project titled **“LegalEase – AI-Powered Legal Document Generator”** is a project documentation prepared for academic submission. The report presents the project through the six phases specified in the program metrics: Brainstorming & Ideation, Requirement Analysis, Project Design, Project Planning using Agile methodologies, Project Development, and Functional & Performance Testing.

Project Guide _____

Head of the Department _____

Internal Examiner _____

External Examiner _____

Date _____

DECLARATION

I hereby declare that this project documentation presents the LegalEase application and its documented implementation components for academic purposes. The report is organized according to the project phase structure supplied for the program and is based on the available project materials.

Place: _____

Date: _____

Signature of Student: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, faculty members, and the institution for providing the opportunity and guidance to complete this project. We also thank the program team for providing a structured project-development process covering ideation, requirements, design, planning, development, and testing. This project helped us understand how a web application, REST API, document-processing utilities, and generative AI can be combined into one practical system.

ABSTRACT

LegalEase is an AI-powered application designed to help users create structured legal-document drafts. The system combines a Streamlit user interface with a FastAPI backend and an AI generation layer identified in the project materials as Google Gemini. Users select a document type, enter parties and terms, provide an effective date, and optionally specify jurisdiction and additional instructions.

After submission, the frontend sends the information to the backend through the **/generate** endpoint. The generated draft is displayed in an editable preview. The final content can be exported as TXT, DOCX, or PDF. The project also includes health-check endpoints and automated tests for basic API behaviour.

The main academic value of LegalEase is its demonstration of an end-to-end software workflow: **User Input** → **Streamlit** → **FastAPI** → **AI Generation** → **Editable Draft** → **Document Export**. The application is intended for AI-assisted drafting; generated legal content should be reviewed by an appropriate legal professional before signing or relying on it.

Keywords: LegalEase, Generative AI, Streamlit, FastAPI, Google Gemini, Python, Document Generation

TABLE OF CONTENTS

Chapter	Title
1	Project Overview
2	Phase 1 – Brainstorming & Ideation
3	Phase 2 – Requirement Analysis
4	Phase 3 – Project Design
5	Phase 4 – Project Planning using Agile Methodologies
6	Phase 5 – Project Development
7	Phase 6 – Functional & Performance Testing
8	Technology Stack and System Requirements
9	Project Structure and API Design
10	Security, Limitations and Future Enhancements
11	Conclusion
12	References and Source Files

Program Metrics

The supplied program-metrics sheet defines six project phases. It also lists a 30-mark technical assessment and a 70-mark project evaluation. The project-evaluation metrics are: Brainstorming & Ideation – 10 marks; Requirement Analysis – 10; Project Design – 15; Project Planning – 5; Project Development – 15; Project Testing – 5; Project Documentation – 5; Project Demonstration – 5.

1. PROJECT OVERVIEW

LegalEase addresses the practical need for structured legal-document drafting through a web-based interface. The project documentation describes support for categories such as Rental Agreement, Employment Agreement, Non-Disclosure Agreement, Service Agreement, Leave and License Agreement, Affidavit, Loan Agreement, Partnership Agreement, and Custom document types.

Project Objectives

- Provide a simple interface for entering legal-document requirements.
- Generate structured document drafts using the configured AI generation service.
- Allow users to edit generated content before export.
- Provide TXT, DOCX and PDF export options.
- Provide a REST API for document generation.
- Provide basic service and health-check endpoints for validation.

Typical User Scenarios

- A startup founder can prepare an employment-contract draft by entering relevant party and employment details.
- A freelancer can provide confidentiality terms to generate an NDA draft.
- A landlord can enter tenant and lease information to prepare a residential lease draft.

2. PHASE 1 – BRAINSTORMING & IDEATION

The first phase focuses on identifying a useful problem and converting the idea into a practical software concept. The LegalEase idea is to use Generative AI to simplify the creation of structured legal-document drafts while giving users the ability to review, edit and export the result.

Problem Identified

Creating a properly structured document manually can require repeated formatting and careful organization of parties, dates, terms and clauses. LegalEase proposes a guided interface where these details are entered in a structured form and converted into a draft.

Idea Formation

- Identify legal-document drafting as the application area.
- Identify Generative AI as the core technology for producing draft content.
- Provide a simple web interface so users can enter information without directly handling code.
- Make the generated result editable before the final export.
- Support common document formats so the output can be reused.

Expected Outcome

A working web application that accepts structured inputs, generates a document draft through the backend/AI workflow, displays the result, and provides downloadable formats.

3. PHASE 2 – REQUIREMENT ANALYSIS

Requirement analysis converts the project idea into clear functional and supporting requirements. The supplied LegalEase interface and backend materials identify the following requirements.

Functional Requirements

- Select a predefined document type or enter a Custom type.
- Enter party information and terms and conditions.
- Provide an effective date.
- Optionally enter jurisdiction and additional instructions.
- Send the request to the FastAPI /generate endpoint.
- Display the generated document in an editable area.
- Download the final content as TXT, DOCX or PDF.
- Provide root and health endpoints for service validation.

Non-Functional / Supporting Requirements

- Simple and user-friendly web interaction.
- Input validation before a generation request is sent.
- Clear handling of connection, timeout and backend errors.
- Modular separation of frontend, backend, schemas, utilities and tests.
- Use of environment configuration for backend/API-related settings.

Inputs and Outputs

Input	Purpose	Output
Document type	Identifies the required document category	Generated document type
Parties	Provides involved-person/entity information	Structured draft
Terms	Provides conditions and requirements	Editable content
Effective date	Sets the document date	TXT/DOCX/PDF

4. PHASE 3 – PROJECT DESIGN

The design phase defines how the frontend, backend, validation layer, AI generation layer and document-export utilities work together. The supplied architecture follows a frontend–backend pattern.

High-Level Architecture

1. Streamlit Frontend	Collects inputs, displays status, shows editable output and provides download controls.
2. FastAPI Backend	Receives requests and exposes /generate, / and /health endpoints.
3. Pydantic Validation	Defines and validates DocumentRequest and DocumentResponse data.
4. AI Generation Layer	Uses the configured Google Gemini generation service to create document content.
5. Export Utilities	Convert final text into TXT, DOCX and PDF formats.

System Workflow

User selects document type → enters parties and terms → selects effective date → optional jurisdiction/instructions → frontend validates input → POST /generate → backend validates request → generation service produces content → result returns to frontend → user edits → TXT/DOCX/PDF export.

Data Model Design

- **DocumentRequest:** document_type, parties, terms, effective_date, jurisdiction, additional_instructions.
- **DocumentResponse:** document_type, content, model.

5. PHASE 4 – PROJECT PLANNING USING AGILE METHODOLOGIES

The program metrics identify Project Planning using Agile methodologies as Phase 4. The supplied project materials do not provide specific sprint dates or individual sprint records, so this documentation presents the phase-wise Agile plan rather than inventing dates.

Agile-Oriented Work Plan

Iteration / Focus	Major Activities	Expected Deliverable
1. Ideation & Requirements	Problem identification, user scenarios, inputs and outputs	Approved project requirements
2. Design	Architecture, API flow, data models and UI planning	System design and module plan
3. Development	Frontend, backend, validation, AI integration and exporters	Working application modules
4. Testing & Refinement	Functional checks, endpoint checks, error handling and export validation	Tested project build

Planning Considerations

- Keep frontend, backend, validation and export modules separated so each part can be developed and checked independently.
- Use clear input fields and validation to reduce incomplete requests.
- Keep document generation behind the backend API so the frontend remains focused on user interaction.
- Test both successful and failure paths before final demonstration.
- Maintain the documentation alongside development so the final report reflects the actual project structure.

Risk and Mitigation

Risk	Mitigation / Handling
Missing required input	Frontend validation before sending the request
Backend unavailable	Display a connection error and request backend startup/retry
Request timeout	Handle timeout separately and ask the user to retry
AI-generated errors	Treat output as a draft and require appropriate human/legal review

6. PHASE 5 – PROJECT DEVELOPMENT

Development converts the approved design into working modules. The LegalEase project uses Python as the primary language and combines Streamlit, FastAPI, Pydantic, Google Gemini, python-docx and FPDF-based PDF generation.

Frontend Development – Streamlit

- Displays the LegalEase title and project description.
- Provides document-type selection and input fields.
- Collects parties, terms, effective date, jurisdiction and additional instructions.
- Sends the JSON request to the configured backend URL.
- Displays the generated result in an editable text area.
- Provides TXT, DOCX and PDF download buttons.

Backend Development – FastAPI

- main.py initializes the FastAPI application and includes the routes.
- routes.py defines the POST /generate document-generation route.
- schemas.py defines DocumentRequest and DocumentResponse using Pydantic.
- The root endpoint reports service status and the /health endpoint supports basic health validation.
- Generation errors are mapped to HTTP error responses.

AI and Document Generation

The project documentation identifies Google Gemini as the AI integration and describes Gemini 1.5 Pro as the selected model in the project workflow. The generation service receives the structured request and returns document content. The internal generator/service source files listed in the README were not included in the supplied source set, so this report does not describe their internal prompt or implementation details.

Export Development

- **TXT:** converts the final content to UTF-8 bytes.
- **DOCX:** creates a Word document and applies Times New Roman styling to the normal document style.
- **PDF:** uses FPDF with automatic page breaks and formatted title/content output.

7. PHASE 6 – FUNCTIONAL & PERFORMANCE TESTING

Testing verifies whether the major functions behave as expected and whether common failure conditions are handled clearly. The supplied test and application materials support endpoint tests, input validation, communication error handling and export checks.

Functional Test Cases

Test ID	Test / Action	Expected Result
TC-01	GET /	HTTP 200 and status is running
TC-02	GET /health	HTTP 200 and status is ok
TC-03	Generate without document type	Required-field validation message
TC-04	Generate without parties	Required-field validation message
TC-05	Generate without terms	Required-field validation message
TC-06	Backend unavailable	Connection error is displayed
TC-07	Generation timeout	Timeout message is displayed
TC-08	Successful generation	Content is displayed and downloadable

Performance and Reliability Considerations

- The frontend includes request timeout handling so a delayed backend response does not remain unexplained.
- The backend separates runtime failures from other generation failures through different HTTP error responses.
- Export functions are tested for TXT, DOCX and PDF conversion.
- No numerical response-time benchmark is supplied in the project materials; therefore no unsupported performance value is reported here.

Testing Outcome

The documented test cases provide coverage for service availability, required inputs, backend communication, generation success, and downloadable output. These checks support the academic demonstration of the application workflow.

8. TECHNOLOGY STACK AND SYSTEM REQUIREMENTS

Technology / Package	Role
Python	Primary programming language
Streamlit	Frontend and web interface
FastAPI	Backend framework and REST API layer
Google Gemini	AI generation integration
Pydantic	Request and response validation
python-docx	DOCX document creation
fpdf2 / FPDF	PDF document creation
Requests	Frontend-to-backend HTTP communication
python-dotenv	Environment configuration
Uvicorn	ASGI server for FastAPI
Pytest	Testing framework

System Requirements

- Python environment capable of installing the project dependencies.
- FastAPI and Uvicorn for backend execution.
- Streamlit for frontend execution.
- Modern web browser for accessing the application.
- Network access may be required for configured AI/API services.
- A computer capable of running Python, the backend and Streamlit application.

9. PROJECT STRUCTURE AND API DESIGN

Project Structure

LegalEase/
■■■ backend/ (main.py, routes.py, schemas.py, services/)
■■■ frontend/ (app.py)
■■■ utils/ (exporters.py, text_utils.py)
■■■ assets/
■■■ tests/ (test_health.py, test_exports.py)
■■■ .env / .env.example / .gitignore
■■■ requirements.txt
■■■ README.md

API Endpoints

Method	Endpoint	Purpose
GET	/	Service root/status endpoint
GET	/health	Health-check endpoint
POST	/generate	Generate a legal-document draft from a validated request

Request and Response

DocumentRequest: document_type, parties, terms, effective_date, jurisdiction, additional_instructions.
DocumentResponse: document_type, content, model.

Responsible Use

The application documentation warns that AI-generated legal documents may contain errors and should be reviewed by a qualified legal professional before signing or relying on them. LegalEase should therefore be understood as an AI-assisted drafting tool rather than a substitute for professional legal advice.

10. SECURITY, LIMITATIONS AND FUTURE ENHANCEMENTS

Security and Responsible Use

- The backend applies CORS configuration for the local Streamlit origins shown in the supplied main.py.
- Environment variables are used for configuration such as the backend URL and AI-related settings.
- The supplied source set does not provide enough detail to describe the internal API-key handling or generation prompt implementation.
- Users should review generated legal content before using it for a real legal purpose.

Current Limitations

- The internal document-generation service and Gemini generator files are listed in the README but were not included in the supplied source set.
- AI-generated content may contain errors and requires human review.
- The supplied PDF exporter replaces unsupported Unicode characters, which may affect some non-Latin text.
- The materials do not demonstrate production deployment, authentication, database storage or user-account management.
- No numerical performance benchmark is included in the supplied materials.

Future Enhancements

- Add user authentication and role-based access where required.
- Add persistent storage for document drafts and revision history.
- Improve PDF Unicode and multilingual font support.
- Add richer templates and stronger domain-specific validation.
- Expand automated tests for generation, API validation and export formats.
- Provide hosted deployment configuration.
- Add a dedicated final-document preview before download.
- Integrate deeper contract analysis, legal databases and personalized recommendations as future project extensions.

11. CONCLUSION

LegalEase demonstrates how Generative AI can be integrated into a practical web application for structured legal-document drafting. The project combines a Streamlit frontend, FastAPI backend, Pydantic data models, an AI generation layer, and document-export utilities into one workflow.

The six-phase project structure provides a clear academic path from idea formation to requirements, design, planning, development and testing. The final workflow allows a user to enter structured information, request a generated draft, edit the returned content, and download the result as TXT, DOCX or PDF.

As an academic project, LegalEase demonstrates frontend–backend integration, API communication, AI-assisted content generation, validation, document formatting and testing. For real-world legal use, the generated content must be treated as a draft and reviewed appropriately.

PROJECT PHASE SUMMARY

Phase	Title	Key Output
1	Brainstorming & Ideation	Problem, idea and user scenarios
2	Requirement Analysis	Functional and supporting requirements
3	Project Design	Architecture, workflow and data models
4	Project Planning	Agile-oriented phase plan and risk handling
5	Project Development	Frontend, backend, AI and export modules
6	Functional & Performance Testing	Test cases, error handling and validation

12. REFERENCES AND SOURCE FILES

This documentation was prepared from the LegalEase project materials supplied for the academic project. The main project description, workflow, architecture, user interaction, testing information and conclusion were taken from the supplied LegalEase documentation and source files.

Source	Use in Documentation
LegalEase.pdf	Project description, workflow, model selection, architecture, core functions, deployment, user interaction and conclusion
README.md	Project overview, technology stack, features, project structure and logo information
frontend/app.py	User interface, input validation, generation request, editing and downloads
backend/main.py	FastAPI application, root/health endpoints and configuration
backend/routes.py	POST /generate route and error mapping
backend/schemas.py	DocumentRequest and DocumentResponse models
utils/exporters.py	TXT, DOCX and PDF export implementation
tests/test_health.py	Root and health endpoint tests
tests/test_exports.py	Export conversion tests
requirements.txt	Python package dependencies
Program Metrics sheet	Six project phases and evaluation metrics used to organize this report

END OF PROJECT DOCUMENTATION